

Narrowing and Widening Conversions

Project:	ISO JTC1/SC22/WG21: Programming Language C++
Number:	D-FIX
Date:	2019-02-23
Audience:	EWGI
Revises:	none
Author:	Lawrence Cowl
Contact	Lawrence@Cowl.org

Abstract

Generalizing narrowing conversions can admit new primitive numeric types without further updates to the standard. Giving priority to select widening conversions in overload resolution will ease the construction and use of overloaded functions.

Contents

TBD

Introduction

Overload resolution relies on finding the best implicit conversion sequence from each argument type to its corresponding parameter type in all candidate functions. When no one function has all the best sequences, the function call is ambiguous and therefore ill-formed.

While many standard conversions may appear in a conversion sequence, only one user-defined conversion may appear.

The standard distinguishes between information-preserving (widening) conversions and information-destroying (narrowing) conversions in two ways. First, the standard promotions are widening conversions. Second, initializer-list initialization defines some conversions as narrowing conversions and permits them only when the source is `constexpr` and the value is within the value set of the destination type.

Problem

Consider the following declarations.

```
float atan2( float, float );
double atan2( double, double );
long double atan2( long double, long double );
```

Of the nine possible call argument-type combinations (3^2), six are ambiguous. However, all combinations have a best overload, which preserves all the information in the arguments and operates at the least cost.

```
int main() {
    float f; double d; long double ld;
    atan2( f, f ); // matches float
    atan2( f, d ); // ambiguous, want double
    atan2( f, ld ); // ambiguous, want long double
    atan2( d, f ); // ambiguous, want double
    atan2( d, d ); // matches double
    atan2( d, ld ); // ambiguous, want long double
    atan2( ld, f ); // ambiguous, want long double
    atan2( ld, d ); // ambiguous, want long double
    atan2( ld, ld ); // matches long double
}
```

The problem extends to user-defined types and functions as well.

```
class cardinal {
    unsigned int c;
public:
    cardinal();
};

class integral {
    int c;
public:
    integral();
    integral( cardinal );
    operator cardinal();
};

class rational {
    integral n, d;
public:
    rational();
    rational( cardinal );
    operator cardinal();
    rational( integral );
    operator integral();
};

cardinal add( cardinal, cardinal );
integral add( integral, integral );
rational add( rational, rational );

int main() {
    cardinal c; integral i; rational r;
    add( c, c ); // matches cardinal
    add( c, i ); // ambiguous, want integral
    add( c, r ); // ambiguous, want rational
    add( i, c ); // ambiguous, want integral
    add( i, i ); // matches integral
    add( i, r ); // ambiguous, want rational
    add( r, c ); // ambiguous, want rational
    add( r, i ); // ambiguous, want rational
    add( r, r ); // matches rational
}
```

A related problem is that adding a new overload into a header may introduce an ambiguity in client code. Such problems may not be found until well after products have shipped.

Workarounds for Library Users

The user of library must necessarily be able to work around such issues.

Explicit Argument Casting

The typical workaround to ambiguity for library users is to add explicit casts to the call sites.

```
int main() {
    atan2( f, f );
    atan2( static_cast<double>(f), d );
    atan2( static_cast<long double>(f), ld );
    atan2( d, static_cast<double>(f) );
    atan2( d, d );
    atan2( static_cast<long double>(d), ld );
}
```

```

    atan2( ld, static_cast<long double>(f) );
    atan2( ld, static_cast<long double>(d) );
    atan2( ld, ld );
}

```

Static Function Declaration

One workaround to this problem is to define a local static function with exactly the needed arguments..

```

static long double atan2( float f, double d )
    return atan2( static_cast<double>(f), d );
}

int main() {
    float f; double d;
    atan2( f, d );
}

```

Local Extern Declaration

One workaround to this problem is Daveed Vandervorde's technique that adds a local extern function declaration to force a particular overload.

```

int main() {
    float f; double d;
    extern long double atan2( double, double );
    return atan2( f, d );
}

```

This technique is effective, but not well known. It lacks generality in that it does not apply to member functions. More importantly, as more overloads are used within the function, the ambiguity problem resurfaces.

Workarounds for Library Authors

Library authors can anticipate some problems.

Add Additional Overloads

The primary workaround is to add more overloaded functions. Unfortunately, argument-dependent lookup problems may arise when the new functions and the original functions are in different namespaces.

```

double atan2( float f, double d )
    { return atan2( static_cast(c), i ); }
long double atan2( float f, long double ld )
    { return atan2( static_cast(c), r ); }
double atan2( double d, float f )
    { return atan2( i, static_cast(c) ); }
long double atan2( double d, long double ld )
    { return atan2( static_cast(i), r ); }
long double atan2( long double ld, float f )
    { return atan2( r, static_cast(c) ); }
long double atan2( long double ld, double d )
    { return atan2( r, static_cast(i) ); }

```

Unfortunately, number of additional overloads needed grows dramatically with increasing number of types and parameters. This growth places a specification burden on the library author. It also places a burden on the library user, because the number of overloads that must be excluded by a call also grows.

Make Lossy Conversions Explicit

Problematic conversion can be excluded from overloading by making them explicit. However, this approach requires casting even when there would otherwise be no ambiguity.

Common Type Template

Another approach to solving the problem is to write templates that convert arguments to a common type (e.g. [P0880R2](#)). Unfortunately this approach has difficulty with argument-dependent lookup and namespaces.

Solution

In the examples above, there is always one least-common information-preserving overload. There are two mechanisms that make identifying this overload possible. First, we prefer conversions that preserve information over those that lose information. Second, we prefer the promotion that covers the least distance.

In [N3387](#), Jens Maurer applied these principles to integer types by adjusting the rules for integer conversion rank and promotion. The paper was not pursued. We intend to generalize the approach to user-defined types.

Proposal

We propose to introduce a distinction between narrowing and widening conversions in C++ programs. We then propose to alter overload resolution rules to prefer widening conversion over narrowing conversions.

For built-in types:

- Generalize the definition of conversion to value sets rather than specific conversion type pairs.
- Define widening standard conversions as those from a lower rank to those with a higher rank where the value set of the former is a subset of the later.
- Modify overload rules to prefer widening standard conversions over other standard conversions.

For user-defined types:

- Add a keyword `widening`, to be used where `explicit` may appear, which declares a user-defined conversion to be a widening conversion.
- Define a widening standard conversion sequence to be a standard conversion sequence that does not have narrowing standard conversions.
- Defining a widening user-defined conversion sequence to be a widening standard conversion sequence, a widening user-defined conversion, and a widening standard conversion sequence.
- Change operator overloading to prefer either:
 - option 1: standard conversion sequence over widening user-defined conversion sequence over non-widening user-defined conversion sequence; or
 - option 2: widening standard conversion sequence over widening user-defined conversion sequence over narrowing user-defined conversion sequence over non-widening user-defined conversion sequence
- Change operator overloading to prefer the 'nearest' widening conversion. The nearest widening conversion is the one that preserves more possible subsequent conversions. Specifically, given widening conversions $A \rightarrow B$, $A \rightarrow C$ and $B \rightarrow C$, but not $C \rightarrow B$, a conversion from A to B preserves a subsequent B to C . On the other hand, a conversion from A to C has no subsequent conversions.

Consequences

The proposal only removes ambiguity, it does not introduce it. So, all existing code is correct and unchanged.

The two's complement representation for builtin integer types, implies signed–unsigned conversions are not widening.

When programmers change existing user-defined conversions to widening conversions, option 1 will not introduce user-defined conversions where none existed before. It is unclear whether option 2 may do so.

The parameter types have widening conversions, adding new overloads reduces the chance that existing calls will become ambiguous.

In the case where an existing call would be ambiguous, overload resolution would become more expensive. This expense must be evaluated with respect to the workarounds.

Wording

All wording edits are relative to [N4800 Working Draft, Standard for Programming Language C++](#).

THE WORDING IS NOT YET COMPLETE.

5.11 Keywords [lex.key]

Add widening to Table 5 — Keywords.

7.3 Standard conversions [conv]

9.3.4 List-initialization [dcl.init.list]

paragraph 7 defines narrowing conversion

10.3.4 Constructors [class.ctor]

11.3.3 Best viable function [over.match.best]

11.3.3.1 Implicit conversion sequences [over.best.ics]

References

[N3387]

[N3387](#) Overload resolution tiebreakers for integer types; 2012-09-12; <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2012/n3387.html>

[P0880R2]

[N3387](#) Numbers interaction; 2019-01-15; <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p0880r2.html>

[N4800]

[N4800](#) Working Draft, Standard for Programming Language C++; 2019-01-21; <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/n4800.pdf>